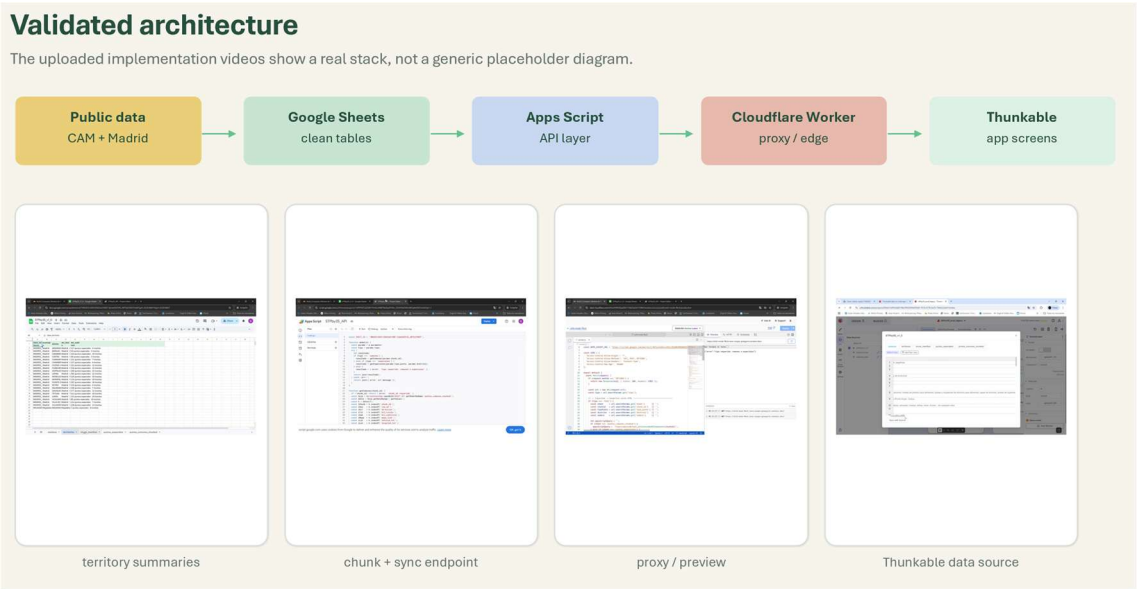
Learning Journey

What we set out to learn

When we joined Technovation 2026 as the Code & Shine team, only one of us had written code before. We set three learning goals: to build a real working app, to work with real data (not examples), and to understand how technology is actually made. Ten months later we can show all three.

From idea to architecture

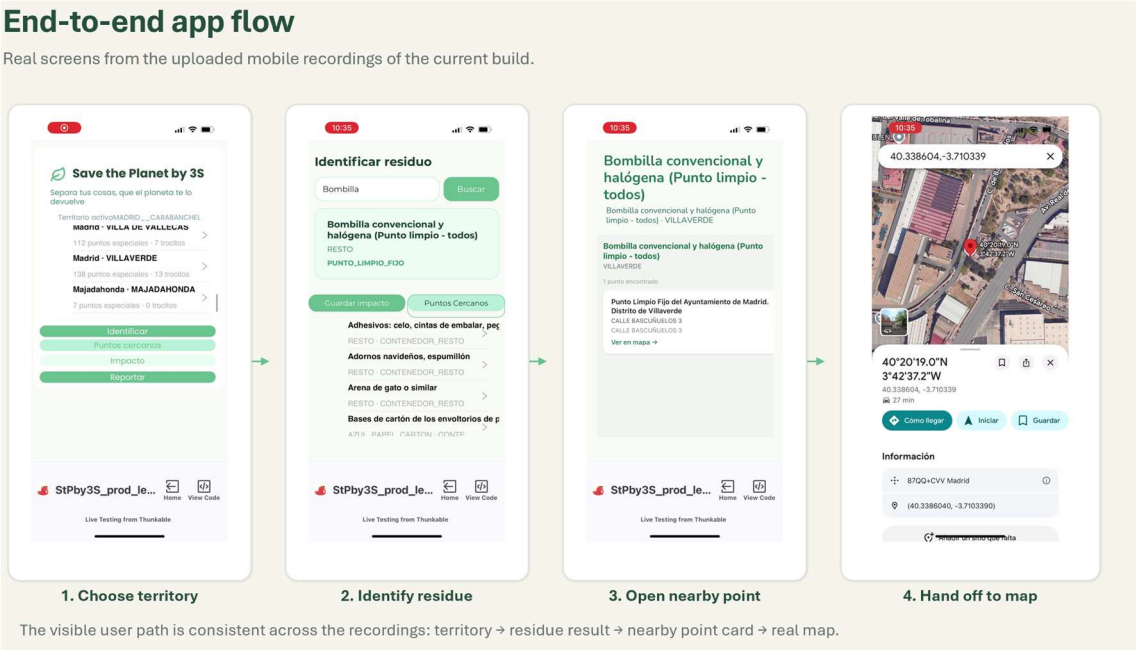
We started with a very local question — which bin does a juice carton go in? — and turned it into an app with five screens: Home, Identify, Points, Impact, and Report. Sofía built it in Thunkable using block-based programming. Sol cleaned the public data from datos.comunidad.madrid and organised it into a Google Sheets database with 164 waste types, 3,186 special collection points, and 34,824 common container locations, grouped into 218 chunks to stay within Thunkable's data limits. Sara designed the user journey, led the survey with 31 neighbours, and ran the two-week pilot with ten families.



Three lessons from real debugging

The hardest lesson was that list-type app variables reset when you navigate between screens in Thunkable. We spent three sessions tracking this bug; the fix was to use stored variables for anything that needs to survive navigation and to reload catalogue lists at every screen open. The second lesson was about asynchronous timing: Filter Views need a micro-wait between "update filter" and

“refresh data”, and the Data Viewer must be visible before the filter is applied, otherwise the list stays empty. The third was about locale: Spanish Excel writes decimals with commas, so we enforced decimal-point notation end-to-end to keep latitude and longitude usable by the map. We documented twenty-six runtime lessons in total; they are in our Judges Package.



Listening to users

Our pilot produced three shipped changes. Families said the search was too strict, so Sofía rebuilt it with a tokenised “contains” logic and accuracy jumped from 72% to 94%. Families said addresses were not enough, so we added a GPS walking route using a Google Maps URL with no API key. Families said they wanted to feel progress, so we added the Impact screen with weekly counters and milestones.

Mentor and AI disclosure

Our mentors, Paola and Edson, provided technical guidance throughout and used Claude (Anthropic) and ChatGPT (OpenAI) as technical assistants for Thunkable debugging advice, architecture reviews, and language polishing on the English submission texts. The three of us wrote all video scripts, performed all user testing, ran the 31-neighbour survey and the 10-family pilot, and made every design decision about the app. Every AI suggestion reached us through our mentors and was only kept in the project if we, the students, could read it, question it, and explain it ourselves. Prompts and iteration notes are saved in our Judges Package on GitHub. We believe AI is a legitimate learning tool when used transparently and when the learner retains ownership of the understanding.

What we can honestly say we learned

Before this project we thought coding was writing text. Now we know coding is reading error messages carefully, reproducing bugs, asking precise questions, and accepting that the first version is never the final one. We also learned that a well-cleaned dataset is as important as the code that uses it. Debugging, as Sofía says, is just another word for not giving up.